claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-ad7593db04ad96294.jsonl`
- SHA-256: `bb84abf27d1c4abb4d3e1b59e64d3cf725d7e442bddd891eea735b4b3aef4e3c`
- Source modified: `2026-06-21T19:15:48+00:00`
- Imported at: `2026-07-08T15:59:55+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:12:53.044Z)

```
You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the
REAL code the owner will deploy.

CRITICAL repo rules:
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is
checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the
working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared
checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'      (search master's tree)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."        (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you
MAY use Read/Grep/Glob directly there.

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line
evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely.
If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note.
Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat;
'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----
CLAIM (§2.2 + §7): `allowed_video_root` is a path-jail REQUIRED when exposed; /api/process
REJECTS any video path outside it; and `upload_dir` MUST sit UNDER allowed_video_root or
uploaded files are rejected.
On origin/master, find where /api/process (or the ingest/validation layer) enforces the jail
(search 'allowed_video_root' across backend/api + backend, via git grep on origin/master).
Confirm: (a) a path outside the jail is rejected; (b) the check resists traversal (`..`,
symlinks, absolute paths) ? read the actual normalization; (c) confirm the upload_dir-under-jail
requirement is real (trace where an uploaded file lands and how /api/process validates it); (d)
does a `gs://` URI bypass the jail by design (cloud ingest) ? is that intended & safe? This is
the exact caveat that the icebreaker warns about ? get it exactly right.
```

2. assistant (2026-06-21T19:12:56.672Z)

```
I'll verify this claim about `allowed_video_root` path-jail enforcement. Let me start by reading
the doc claim in khelsutra, then check the engine's origin/master.
```

3. user (2026-06-21T19:12:58.311Z)

Deploy KhelSutra Studio for alpha ? a shareable page, no SSH

> **Status:** `READY` (owner executes the cloud steps) · **Topology:** split ? SPA on Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access are free ?50 users) + Gemini per-run (~$0.05/match).

This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a **page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.

0. The shape

```
 tester browser
    ? https
    ?
 Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH hostnames
    ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA; built with VITE_API_BASE)
    ??? api.khelsutra.guru  ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on YOUR box
                                                                    ?  FastAPI engine
                                                                    ?  (Gemini segmenter; GPU
optional)
                                                                    ?  async jobs, SQLite,
output/ on disk
```

- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`. The box's home/public IP is never exposed.
- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to the tunnel can't spoof identity.
- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-driven).

> **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see `LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose (stable Pages-hosted page + a thin API tunnel).

---

1. Prerequisites

- A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU; fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed; the `badminton-highlight-indexer` engine checked out.
- A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing site there).
- `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/`).
- A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks to Google; this is a paid + cloud action).

---

2. Engine (the `api.` side)

2.1 Install (with the auth extra)

```bash
cd badminton-highlight-indexer
```

```
pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every
request 500s
```

2.2 The safe-exposed config

Create / edit `config.json` on the box:

```jsonc
{
  "security": {
    "edge_auth_enabled": true,                        // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming",  // the path-jail ? REQUIRED when exposed
    "upload_dir": "/srv/khelsutra/incoming/uploads",  // MUST sit UNDER allowed_video_root (see
§7)
    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
    "cf_access_aud": "<the CF Access application AUD>"  // filled in §4
  }
}
```

And export the env (a `.env` or your shell):

```bash
export GEMINI_API_KEY="<your-rotated-key>"
export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA origin
```

> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.

2.3 Run

```bash
python -m backend.main --server     # binds 127.0.0.1:8000 (the safe default ? do NOT change the
bind)
```

Keep it alive with a process manager (`systemd` on the droplet, mirroring `site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

---

3. The SPA on Cloudflare Pages (the `app.` side)

The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-deploy on merge to `master`).

1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick the `khelsutra` repo.
2. **Build settings:**
   - **Root directory:** `web`
   - **Build command:** `npm run build`
   - **Build output directory:** `web/dist`
   - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this is what points the SPA at the engine tunnel (PR #64).
3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

> Every push to `master` now rebuilds + redeploys the SPA. To preview a change first, Pages builds a per-PR preview URL.

---

4. Cloudflare Access (gate BOTH hostnames)

1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the

SPA and its API calls are authorized).
3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

---

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                  # browser auth to your zone
cloudflared tunnel create khelsutra-studio                # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNE
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
cloudflared tunnel run khelsutra-studio                   # or install as a service: `cloudflared service install`
```

See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).

---

6. Verify (the posture test + the CUJ)

Exposure posture (must hold ? this is what makes it safe to share):
- From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?

The shareable CUJ (what a tester does): open `app.khelsutra.guru` ? sign in ? paste a `gs://?` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page.

---

7. Honest caveats / deferred

- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
- **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
- **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and ingest the `gs://` URI (already supported).
- **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this with a queue (`HOSTING_PLAN.md` §3).
- **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for clips

you're OK sending.

---

8. Teardown

`cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages custom domain; disable the Access application; stop the engine service. The box's `output/` + `uploads/` retain originals ? sweep them per your retention policy before any wider exposure.

Changelog

- 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative) + the engine `HOSTING_PLAN.md` §2.

4. user (2026-06-21T19:12:59.434Z)

```
origin/master:backend/config/models.py:283:    allowed_video_root: Optional[str] = None
origin/master:backend/config/models.py:287:    # arrive with PR-3 identity scoping. ? In hosted
mode, set allowed_video_root to
origin/master:backend/config/startup.py:22:rest of the posture at **boot**: the
``allowed_video_root`` path-jail is set (else an authenticated
origin/master:backend/config/startup.py:104:    # a tenant can point the engine at an ARBITRARY
local file (allowed_video_root unset = "any local
origin/master:backend/config/startup.py:106:    root = getattr(sec, "allowed_video_root", None)
origin/master:backend/config/startup.py:110:            "security.edge_auth_enabled is ON
(engine exposed) but security.allowed_video_root is "
origin/master:backend/config/startup.py:112:            "Set allowed_video_root to a jail
directory before exposing the engine.",
origin/master:backend/main.py:575:    allowed_root = getattr(getattr(global_config, "security",
None), "allowed_video_root", None)
origin/master:backend/main.py:587:    # input: its null-byte guard always applies, and a
configured allowed_video_root (hosted mode)
origin/master:backend/main.py:849:    auth on without the ``allowed_video_root`` jail / CF
team+AUD). A strict no-op when no
```

5. assistant (2026-06-21T19:13:03.479Z)

The doc claim has three parts. Let me read the actual enforcement code on origin/master. The key file appears to be `backend/main.py` around lines 575-590 and `backend/config/startup.py`.

6. user (2026-06-21T19:13:04.893Z)

```
            "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.",
            "results": [r.model_dump() for r in val_results],
            "play_segments": segments,
        }
        return response
    finally:
        # Always emit the per-video observability report (next to the video, or to
        # report.output_dir). Never raises. Surface the paths in the response.
        paths = emit_indexer_report(
            db=db_instance, video_id=video_id, video_path=req.video_path, config=global_config,
            val_results=val_results, val_context=val_context,
            segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
            was_reingest=was_reingest, segmentation_ran=segmentation_ran,
        )
        if paths and isinstance(response, dict):
            response["reports"] = paths


@app.post("/api/process", status_code=202)
```

```
def process_video(req: ProcessRequest, request: Request):
    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
    client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
    runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
    """
    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
    # spoofed identity header fails here with 401, before any work) and tags the job's owner_id.
    try:
        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root", None)
    try:
        validate_input_path(req.video_path, allowed_root=allowed_root)
        # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
        # scheme, which we map to a clean 400 (an unknown scheme must never 500).
        input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config, "storage",
None))
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    # The local-existence fast-fail applies only to a LOCAL input, and stats the RESOLVED local
path
    # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI string. A
    # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified when the
worker
    # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs for
every
    # input: its null-byte guard always applies, and a configured allowed_video_root (hosted
mode)
    # rejects a non-local URI as out-of-root.
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
    if req.segmenter and not segmenter_registry.get(req.segmenter):
        available = list(segmenter_registry.list_available().keys())
        raise HTTPException(
            status_code=400,
            detail=f"Segmenter '{req.segmenter}' not found. Available segmenters: {available}"
        )

    job_id = uuid.uuid4().hex
    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                  req.player_pool, req.max_frames, owner_id=owner_id,
                                  locale=req.locale):
        raise HTTPException(status_code=500, detail="Failed to persist job record.")
    _get_executor().submit(_drain_due_jobs)
    return JSONResponse(
        status_code=202,
        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
    )


@app.get("/api/jobs/{job_id}/cost")
def get_job_cost(job_id: str):
    """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of truth
(matches
    GCP billing); ``cost_inr`` is a display conversion at the configured FX rate. ``cost_usd``
is
    null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
```

```
    cost = db_instance.get_job_cost(job_id)
    if cost is None:
        raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
    fx = global_config.billing.fx_inr_per_usd
    usd = cost.get("cost_usd")
    cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
    cost["fx_inr_per_usd"] = fx
    return cost


@app.get("/api/videos/{video_id}/cost")
def get_video_cost(video_id: str):
    """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` + ``total_cost_inr`` +
the
    per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce several."""
    agg = db_instance.get_video_cost(video_id)
    fx = global_config.billing.fx_inr_per_usd
    agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
    agg["fx_inr_per_usd"] = fx
    return agg


@app.get("/api/jobs/{job_id}")
def get_job(job_id: str):
    """Job state: {status, progress, result, reports}. `status` = execution state
    (queued|running|done|failed); the pipeline verdict is result["status"]."""
    job = db_instance.get_job(job_id)
    if not job:
        raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
    result = job.get("result")
    return {
        "job_id": job["id"],
        "status": job["status"],
        "progress": job.get("progress"),
        "video_id": job.get("video_id"),
        "error": job.get("error"),
        "result": result,
        "reports": (result or {}).get("reports"),
        "created_at": job.get("created_at"),
        "started_at": job.get("started_at"),
        "finished_at": job.get("finished_at"),
    }
```

--- Chunked upload (B2, PR-2) ----------

**The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
backend.main.global_config, so behavior ? route paths, status codes, body-size limits
sequential-part logic, and abort cleanup ? is unchanged.**

```
from backend.api import uploads as uploads_api

app.include_router(uploads_api.router)
```

--- Highlight download (B3, PR-2) ----------

```
@app.get("/api/highlights/{video_id}/{filename}")
def download_highlight(video_id: str, filename: str):
    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
    (which only writes into output_dir; the browser previously got back an unreachable
    server-local path)."""
    try:
```

```
        name = safe_output_filename(filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    if not db_instance.get_video(video_id):
        raise HTTPException(status_code=404, detail="Indexed video record not found.")
    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
```

7. assistant (2026-06-21T19:13:08.522Z)

```
Now let me find the `validate_input_path` implementation, which is the core of the jail
enforcement.
```

8. user (2026-06-21T19:13:10.278Z)

```
origin/master:backend/utils/paths.py:53:def validate_input_path(path: str, allowed_root:
Optional[str] = None) -> None:
```

9. user (2026-06-21T19:13:11.206Z)

```
origin/master:backend/config/models.py:283:    allowed_video_root: Optional[str] = None
origin/master:backend/config/models.py:287:    # arrive with PR-3 identity scoping. ? In hosted
mode, set allowed_video_root to
origin/master:backend/config/startup.py:22:rest of the posture at **boot**: the
``allowed_video_root`` path-jail is set (else an authenticated
origin/master:backend/config/startup.py:104:    # a tenant can point the engine at an ARBITRARY
local file (allowed_video_root unset = "any local
origin/master:backend/config/startup.py:106:    root = getattr(sec, "allowed_video_root", None)
origin/master:backend/config/startup.py:110:            "security.edge_auth_enabled is ON
(engine exposed) but security.allowed_video_root is "
origin/master:backend/config/startup.py:112:            "Set allowed_video_root to a jail
directory before exposing the engine.",
origin/master:backend/main.py:45:from backend.utils.paths import resolve_under_root,
safe_output_filename, validate_input_path
origin/master:backend/main.py:575:    allowed_root = getattr(getattr(global_config, "security",
None), "allowed_video_root", None)
origin/master:backend/main.py:577:        validate_input_path(req.video_path,
allowed_root=allowed_root)
origin/master:backend/main.py:586:    # fetches it (a fetch miss fails the job loudly).
validate_input_path above still runs for every
origin/master:backend/main.py:587:    # input: its null-byte guard always applies, and a
configured allowed_video_root (hosted mode)
origin/master:backend/main.py:849:    auth on without the ``allowed_video_root`` jail / CF
team+AUD). A strict no-op when no
origin/master:backend/utils/paths.py:53:def validate_input_path(path: str, allowed_root:
Optional[str] = None) -> None:
origin/master:backend/utils/paths.py:54:    """Reject null bytes and (when an allowed_root is
configured) out-of-root inputs.
origin/master:backend/utils/paths.py:56:    With no allowed_root (default, single-user local
mode) only the null-byte check applies,
origin/master:backend/utils/paths.py:58:    allowed_root (a future hosted/tenant deployment)
enforces containment.
origin/master:backend/utils/paths.py:62:    if allowed_root:
origin/master:backend/utils/paths.py:63:        resolve_under_root(path, allowed_root)
```

10. assistant (2026-06-21T19:13:13.675Z)

```
Now let me read the full `paths.py` to understand the normalization, traversal resistance, and
how `gs://` URIs are handled.
```

11. user (2026-06-21T19:13:15.032Z)

```
"""Path-safety helpers for the API boundary (untrusted request fields)."""
import os
```

```
from typing import Any, Optional


def output_base(config: Any) -> str:
    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
    (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
    chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir``
instead of
    a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior literal.

    This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be
a
    ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy dict-shim
    (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
    try:
        out = config.get("output", {}).get("output_dir")
    except Exception:
        out = None
    return out or "output"


def safe_output_filename(name: str) -> str:
    """Validate an output filename is a BARE name (no dir part, no traversal, not absolute).

    `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is absolute,
    and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-write
    primitive. Returns the name unchanged when safe; raises ValueError otherwise.
    """
    if not name or not name.strip():
        raise ValueError("output_filename must not be empty")
    if "\x00" in name:
        raise ValueError("output_filename must not contain a null byte")
    if "/" in name or "\\" in name:
        raise ValueError(f"output_filename must not contain a path separator: {name!r}")
    if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
        raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
    return name


def resolve_under_root(path: str, root: str) -> str:
    """Return the real path, asserting it is contained under `root` (raises ValueError if
not)."""
    rp = os.path.realpath(path)
    rr = os.path.realpath(root)
    try:
        contained = os.path.commonpath([rp, rr]) == rr
    except ValueError:  # different drives on Windows
        contained = False
    if not contained:
        raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
    return rp


def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
    """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.

    With no allowed_root (default, single-user local mode) only the null-byte check applies,
    so the local 'point it at any of my videos' workflow is preserved. Setting an
    allowed_root (a future hosted/tenant deployment) enforces containment.
    """
    if "\x00" in path:
        raise ValueError("video_path must not contain a null byte")
    if allowed_root:
        resolve_under_root(path, allowed_root)
```

12. assistant (2026-06-21T19:13:19.537Z)

This is the heart of the jail. Now let me verify the `gs://` URI behavior ? the doc claims `gs://` ingest is "already supported" and bypasses the jail by design. Let me trace `resolve_under_root` against a `gs://` path, and check `resolve_input_ref` ordering.

The critical detail: in `main.py`, `validate_input_path(req.video_path, allowed_root=allowed_root)` runs FIRST, then `storage.resolve_input_ref(...)`. So `resolve_under_root` runs on the raw `gs://...` string when `allowed_root` is set. Let me check what `os.path.realpath("gs://bucket/x.mp4")` does and whether it would escape the root.

13. user (2026-06-21T19:13:22.876Z)

'gs://mybucket/video.mp4' -> realpath=
'C:\\Users\\avidu\\AppData\\Local\\Temp\\gs:\\mybucket\\video.mp4' | contained= False
'/srv/khelsutra/incoming/uploads/a.mp4' -> realpath=
'C:\\srv\\khelsutra\\incoming\\uploads\\a.mp4' | contained= True
'/srv/khelsutra/incoming/../etc/passwd' -> realpath= 'C:\\srv\\khelsutra\\etc\\passwd' |
contained= False
'/etc/passwd' -> realpath= 'C:\\etc\\passwd' | contained= False
'gs://x/../../../etc/passwd' -> realpath= 'C:\\Users\\avidu\\AppData\\Local\\etc\\passwd' |
contained= False

14. assistant (2026-06-21T19:13:29.612Z)

This confirms the key behaviors (Windows `realpath` resolves to local paths, but the containment logic is identical on POSIX). Crucial finding: **when `allowed_root` is set, a `gs://` URI is treated as a local relative path by `os.path.realpath`, lands OUTSIDE the jail, and `resolve_under_root` raises `ValueError` ? 400.** So `gs://` does NOT bypass the jail by design ? it is REJECTED when `allowed_video_root` is set.

The doc's own inline comment in `main.py:587` confirms this: "a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root." But the doc DEPLOY_ALPHA.md §6 and §7 say testers can paste a `gs://` URI AND it's "already supported." This is a real discrepancy. Let me verify the exact ordering and that `resolve_input_ref` never runs for `gs://` when the jail is on.

15. user (2026-06-21T19:13:31.810Z)

<persisted-output>
Output too large (64.6KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\tool-results\bpdma2zkr.txt

Preview (first 2KB):
origin/master:backend/compute/base.py:26:   video_uri: str   # input (gs://? for cloud; a path/local:// for local)
origin/master:backend/compute/base.py:27:   out_uri: str   # output ROOT (gs://? bucket for cloud)
origin/master:backend/compute/cloud_run.py:145:   # would silently return {} for every gs://|s3:// object (the only backends this shim targets).
origin/master:backend/config/models.py:324:   # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` + "x.mp4"
origin/master:backend/config/models.py:325:   # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with ``input_backend``
origin/master:backend/config/models.py:327:   # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are fetched to a
origin/master:backend/config/models.py:340:   # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root directly.
origin/master:backend/main.py:758:   # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://, gdrive://) for a
origin/master:backend/main.py:762:   # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a gs:// path.
origin/master:backend/storage/__init__.py:38:def resolve_input_ref(raw: str, storage_config: Any

```
= None) -> StorageRef:
origin/master:backend/storage/__init__.py:64:    byte-for-byte). A bucket / Drive source
(``gs://``/``s3://``/``gdrive://``, or a bare name under a
origin/master:backend/storage/ref.py:8:    gcs://<bucket>/<key>        -> gcs      (gs://
accepted, normalized to gcs://)
origin/master:backend/storage/ref.py:75:      1. An explicit ``scheme://`` on ``raw``
(``gs://``, ``s3://``, ``local://``, ``gdrive://``) ? as-is.
origin/master:backend/storage/ref.py:78:         (``"gs://bucket/videos"`` + ``"x.mp4"`` ?
``"gs://bucket/videos/x.mp4"``).
...
</persisted-output>
```

16. user (2026-06-21T19:13:32.625Z)

```
    try:
        validate_input_path(req.video_path, allowed_root=allowed_root)
        # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
        # scheme, which we map to a clean 400 (an unknown scheme must never 500).
        input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config, "storage",
None))
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    # The local-existence fast-fail applies only to a LOCAL input, and stats the RESOLVED local
path
    # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI string. A
    # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified when the
worker
    # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs for
every
    # input: its null-byte guard always applies, and a configured allowed_video_root (hosted
mode)
    # rejects a non-local URI as out-of-root.
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
    if req.segmenter and not segmenter_registry.get(req.segmenter):
        available = list(segmenter_registry.list_available().keys())
```

17. assistant (2026-06-21T19:13:38.145Z)

Confirmed. The ordering is `validate_input_path(...)` FIRST (which calls `resolve_under_root` when `allowed_root` is set), and only THEN `resolve_input_ref`. So when the jail is on, a `gs://` URI hits `os.path.realpath` ? resolves outside the jail ? `ValueError` ? 400, BEFORE `resolve_input_ref` ever sees it. The engine's own comment at main.py:587 says exactly this: "a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root."

Now let me verify the other parts of the claim:
1. The upload_dir-under-jail trace ? where does an uploaded file land, and how does `/api/process` validate it.
2. The startup posture (boot fail-fast).
3. Config defaults.

18. user (2026-06-21T19:13:40.782Z)

```
"""Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
under security.upload_dir, then submitted to /api/process by the returned path.
Upload state is in-process by design (this is the single-box alpha): a server restart
aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
arrives with PR-3 identity scoping.
```

```
The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
reads the live value the server set at startup (and so tests that monkeypatch
``backend.main.global_config`` flow through unchanged).
"""
import os
import threading
import uuid
from typing import Any, Dict, Optional

from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import safe_output_filename

router = APIRouter()

_uploads: Dict[str, Dict[str, Any]] = {}
_uploads_lock = threading.Lock()


class UploadInitRequest(BaseModel):
    filename: str
    total_size_bytes: Optional[int] = None


class UploadCompleteRequest(BaseModel):
    total_parts: int


def _upload_cfg():
    # Resolved lazily (import inside the function) to read the LIVE startup config that
    # main() sets on the module global, and to avoid a circular import at module load
    # (main.py imports this router; this module must not import main at import time).
    import backend.main as _main
    sec = getattr(_main.global_config, "security", None)
    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
    return upload_dir, max_bytes, part_bytes, exts


def _abort_upload(upload_id: str) -> None:
    with _uploads_lock:
        st = _uploads.pop(upload_id, None)
    if st:
        try:
            os.remove(st["tmp_path"])
        except OSError:
            pass


@router.post("/api/upload/init")
def upload_init(req: UploadInitRequest):
    """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
    try:
        name = safe_output_filename(req.filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    ext = os.path.splitext(name)[1].lower().lstrip(".")
    if ext not in exts:
```

```
        raise HTTPException(status_code=400,
                            detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}")
    if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
        raise HTTPException(status_code=413,
                            detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload
limit.")
    os.makedirs(upload_dir, exist_ok=True)
    upload_id = uuid.uuid4().hex
    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
    open(tmp_path, "wb").close()
    with _uploads_lock:
        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes": 0}
    return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0"}


@router.put("/api/upload/{upload_id}/part/{index}")
async def upload_part(upload_id: str, index: int, request: Request):
    """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
    _, max_bytes, part_bytes, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if index != st["parts"]:
        raise HTTPException(status_code=409,
                            detail=f"Out-of-order part: expected {st['parts']}, got {index}
(parts are sequential)")

    received = 0
    overflow = None
    with open(st["tmp_path"], "ab") as fh:
        async for chunk in request.stream():
            received += len(chunk)
            if received > part_bytes:
                overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                break
            if st["bytes"] + received > max_bytes:
                overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                break
            fh.write(chunk)
    if overflow:
        _abort_upload(upload_id)
        raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")

    with _uploads_lock:
        st["parts"] += 1
        st["bytes"] += received
    return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
```

19. user (2026-06-21T19:13:42.028Z)

```
origin/master:backend/api/uploads.py:5:under security.upload_dir, then submitted to /api/process
by the returned path.
origin/master:backend/api/uploads.py:7:aborts partial uploads and the client re-inits. Per-user
partitioning of upload_dir
origin/master:backend/api/uploads.py:47:    upload_dir = getattr(sec, "upload_dir", None) or
"output/uploads"
origin/master:backend/api/uploads.py:51:    return upload_dir, max_bytes, part_bytes, exts
origin/master:backend/api/uploads.py:67:    upload_dir, max_bytes, part_bytes, exts =
_upload_cfg()
origin/master:backend/api/uploads.py:79:    os.makedirs(upload_dir, exist_ok=True)
origin/master:backend/api/uploads.py:81:    tmp_path = os.path.join(upload_dir,
```

```
f".partial-{upload_id}")
origin/master:backend/api/uploads.py:126:    upload_dir, _, _, _ = _upload_cfg()
origin/master:backend/api/uploads.py:134:    final_path = os.path.join(upload_dir,
st["filename"])
origin/master:backend/api/uploads.py:137:        final_path = os.path.join(upload_dir,
f"{stem}-{upload_id[:8]}{ext}")
origin/master:backend/config/models.py:283:    allowed_video_root: Optional[str] = None
origin/master:backend/config/models.py:287:    # arrive with PR-3 identity scoping. ? In hosted
mode, set allowed_video_root to
origin/master:backend/config/models.py:288:    # contain upload_dir or /api/process will reject
the uploaded paths.
origin/master:backend/config/models.py:289:    upload_dir: str = "output/uploads"
```

20. assistant (2026-06-21T19:13:44.847Z)

Now let me see the rest of uploads.py (the complete endpoint that returns the final_path) and confirm the upload flow returns a path the SPA then POSTs to `/api/process`.

21. user (2026-06-21T19:13:46.598Z)

```
    return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


@router.post("/api/upload/{upload_id}/complete")
def upload_complete(upload_id: str, req: UploadCompleteRequest):
    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
    upload_dir, _, _, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if st["parts"] == 0 or st["parts"] != req.total_parts:
        raise HTTPException(status_code=409,
                            detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
    final_path = os.path.join(upload_dir, st["filename"])
    if os.path.exists(final_path):  # never overwrite an earlier upload with the same name
        stem, ext = os.path.splitext(st["filename"])
        final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
    os.replace(st["tmp_path"], final_path)
    with _uploads_lock:
        _uploads.pop(upload_id, None)
    video_id = compute_video_id(final_path)
    return {"path": os.path.abspath(final_path), "video_id": video_id,
            "size_bytes": os.path.getsize(final_path),
            "next": "POST /api/process with this path"}


@router.delete("/api/upload/{upload_id}")
def upload_abort(upload_id: str):
    """Abort an in-flight upload and delete its partial file."""
    with _uploads_lock:
        known = upload_id in _uploads
    if not known:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    _abort_upload(upload_id)
    return {"status": "aborted", "upload_id": upload_id}
```

22. user (2026-06-21T19:13:47.709Z)

```
    # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
    # them to WARNING; set true to restore full chatter for request-flow debugging.
```

```
    verbose_http: bool = False


class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]
```

23. assistant (2026-06-21T19:13:54.290Z)

This fully confirms the upload_dir-under-jail mechanism:
- `upload_complete` returns `{"path": os.path.abspath(final_path), ...}` where `final_path = os.path.join(upload_dir, st["filename"])`.
- The SPA then POSTs that absolute path to `/api/process`.
- `/api/process` runs `validate_input_path(path, allowed_root)` ? `resolve_under_root` ? if `upload_dir` is NOT under `allowed_video_root`, the path escapes the jail ? `ValueError` ? 400.
So the upload is rejected at process time (a "functional break," exactly as the doc says).
- The config comment at models.py:287-288 explicitly states: "In hosted mode, set allowed_video_root to contain upload_dir or /api/process will reject the uploaded paths."

Now let me verify the boot fail-fast posture (startup.py) to confirm `allowed_video_root` is REQUIRED when exposed.

24. user (2026-06-21T19:13:56.261Z)

```
    """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it (find_spec),
    so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it deterministically
    (whether PyJWT is installed in the test env must not change the WARN's presence)."""
    try:
        import importlib.util
        return importlib.util.find_spec("jwt") is not None
    except Exception:  # pragma: no cover - find_spec only raises on a broken parent package
        return False


def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
    """Exposure-safety posture for an *exposed* HTTP server. Gated on ``security.edge_auth_enabled``
    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I am
    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough: verify the
    rest of the posture at boot so a half-configured exposure fails fast instead of leaking / 500-ing
    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env`` is the
    caller's environment view (so the host-allowlist check is testable)."""
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to today).

    out: list[StartupCheck] = []
```

```
    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process: WITHOUT
it,
    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any
local
    # file", the single-user default). Fatal for an exposed engine.
    root = getattr(sec, "allowed_video_root", None)
    if not root or not str(root).strip():
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
            "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root
is "
            "unset ? an authenticated tenant could make /api/process read an arbitrary local
file. "
            "Set allowed_video_root to a jail directory before exposing the engine.",
        ))

    # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails
on the
    # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
    # "healthy" then 401s every request.
    team = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
    if not team or not aud:
        missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if
not v)
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
            f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access
JWT "
            "cannot be verified (every request would fail). Set security.cf_team_domain "
            "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF
Access "
            "application AUD tag).",
        ))

    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If
it's
    # missing, every request 500s at verify time ? a boot WARN beats a per-request 500. Warning,
not
    # fatal: the auth module's own ImportError is the final authority.
    if not _auth_extra_available():
        out.append(StartupCheck(
            LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
            "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
            "verification will fail at request time. Install the 'auth' extra (pip install -e
.[auth]).",
        ))

    # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding) guard
defaults to
```

25. assistant (2026-06-21T19:14:02.257Z)

```
Now let me confirm this is wired to actually FAIL boot (raise / exit), not just warn, when
LEVEL_ERROR checks fire.
```

26. user (2026-06-21T19:14:04.334Z)

```
"""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).

A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a coherent
runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
(expensive) job runs ? when the active profile is inconsistent with the environment:

  * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one config bug we
```

```
    can detect with certainty + zero false-positive risk: a cloud profile that can't reach a
    bucket can never serve). This is the README §4.3 "loud error before any job".
  * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
    falling back to CPU) ? a **warning**, never an error: the probe is best-effort (torch may
live
    only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
    own healthcheck is the real authority. A warning surfaces it early without false-failing.
  * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on Cloud Run)
?
    a **warning**: Application Default Credentials can come from the metadata server / gcloud,
which
    we can't probe offline, so the storage backend's own init is the authority; we just flag it.

**Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is
*exposed*
behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the Cloudflare-Access
boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety
story.
A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we verify
the
rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are present
(else
no token can verify ? **error**, lifting the current first-request failure to boot), and the
``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP **server**
passes
``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant paths), so it
is
never "exposed" and stays unaffected.

**DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the profile
checks are
a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False`` (today's
default)
the exposure checks are a strict **no-op** too. So nothing changes for today's manual-knob
deployments.
"""

from __future__ import annotations

import logging
import os
from dataclasses import dataclass
from typing import Any, Mapping, Optional

LEVEL_ERROR = "error"
LEVEL_WARN = "warn"
```

Storage backends that count as "a cloud bucket" for cloud-serving coherence.

```
_CLOUD_STORAGE = ("gcs", "s3")
```

STRONG env signals that Google ADC *might* resolve. Deliberately NOT GOOGLE_CL
CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
with no active login), so they would suppress the heads-up falsely. (s3 creds are inten
left to boto3's default chain ? no parallel heads-up, by design.)

```
_GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")


@dataclass(frozen=True)
```

```
class StartupCheck:
    """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-up)."""

    level: str
    code: str       # stable, greppable code (e.g. "STORAGE_INCOHERENT")
    message: str


class StartupCheckError(RuntimeError):
    """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL check."""

    def __init__(self, failures: list[StartupCheck]):
        self.failures = failures
        super().__init__(
            "deployment profile startup check failed: "
            + "; ".join(f"[{c.code}] {c.message}" for c in failures)
        )


def _creds_discoverable(env: Mapping[str, str]) -> bool:
    """True if *some* Google ADC source is hinted by the environment. Best-effort ? a False here
    only downgrades to a WARNING, never a hard failure (the storage init is the authority)."""
    return any(env.get(k) for k in _GCP_ENV_HINTS)


def _auth_extra_available() -> bool:
    """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
```

27. assistant (2026-06-21T19:14:07.324Z)

Now let me confirm `enforce_startup_checks` actually raises and that the server calls it with `include_exposure=True`, so the boot genuinely fails fast (the doc claims "the engine refuses to start unsafe").

28. user (2026-06-21T19:14:09.876Z)

```
origin/master:backend/config/__init__.py:22:    StartupCheckError,
origin/master:backend/config/__init__.py:41:    "StartupCheckError",
origin/master:backend/config/startup.py:18:**Exposure-safety posture (server only,
``include_exposure=True``).** When the engine is *exposed*
origin/master:backend/config/startup.py:26:``include_exposure=True``; the worker reads
``--video-uri`` (not untrusted tenant paths), so it is
origin/master:backend/config/startup.py:62:class StartupCheckError(RuntimeError):
origin/master:backend/config/startup.py:159:    include_exposure: bool = False,
origin/master:backend/config/startup.py:165:    ``include_exposure`` adds the exposure-safety
posture (``_exposure_checks``) ? the HTTP **server**
origin/master:backend/config/startup.py:175:    if include_exposure:
origin/master:backend/config/startup.py:253:def enforce_startup_checks(
origin/master:backend/config/startup.py:259:    include_exposure: bool = False,
origin/master:backend/config/startup.py:263:    ``raise_on_error=False`` ? raise
``StartupCheckError`` if any FATAL check tripped.
origin/master:backend/config/startup.py:265:    ``include_exposure`` (server-only) adds the
exposure-safety posture. Returns the full findings
origin/master:backend/config/startup.py:270:        config, cuda_available=cuda_available,
env=env, include_exposure=include_exposure)
origin/master:backend/config/startup.py:278:        raise StartupCheckError(errors)
origin/master:backend/main.py:54:    StartupCheckError,
origin/master:backend/main.py:848:    the exposed server (``include_exposure=True``, M9
hardening), a half-configured exposure (edge
origin/master:backend/main.py:854:        enforce_startup_checks(config, logger=logger,
include_exposure=True)
origin/master:backend/main.py:855:    except StartupCheckError as e:
origin/master:backend/worker/__main__.py:30:    StartupCheckError,
origin/master:backend/worker/__main__.py:111:        enforce_startup_checks(config,
```

```
cuda_available=cuda, logger=logger)
origin/master:backend/worker/__main__.py:113:    except StartupCheckError:
```

29. user (2026-06-21T19:14:13.247Z)

```
        print("  [ERROR] No visual frames were extracted in the sample range.")

    print("=" * 60)


def _enforce_startup_or_exit(config) -> None:
    """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
BEFORE
    any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only storage) OR,
for
    the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
(edge
    auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
    ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU probe
here ?
    the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so
the
    fail-fast wiring + the default-OFF no-op are unit-testable."""
    try:
        enforce_startup_checks(config, logger=logger, include_exposure=True)
    except StartupCheckError as e:
        logger.error("[STARTUP] refusing to start: %s", e)
        sys.exit(1)


def main():
    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip
to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
    parser.add_argument("--app", action="store_true",
                        help="Launch the app: start the local server (localhost-only) and open
the "
                             "bundled UI in your browser. Falls back to a free port if --port is
busy.")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
on")
```

30. user (2026-06-21T19:14:14.047Z)

```
def enforce_startup_checks(
    config: Any,
    *,
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    raise_on_error: bool = True,
    include_exposure: bool = False,
    logger: Optional[logging.Logger] = None,
) -> list[StartupCheck]:
    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

    ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full
findings
```

```
    list (so a caller can inspect/surface them). A no-op with no logging and no raise when both
    dimensions are off (``run_startup_checks`` returns ``[]``)."""
    log = logger or logging.getLogger(__name__)
    checks = run_startup_checks(
        config, cuda_available=cuda_available, env=env, include_exposure=include_exposure)
    for c in checks:
        if c.level == LEVEL_ERROR:
            log.error("[STARTUP] %s: %s", c.code, c.message)
        else:
            log.warning("[STARTUP] %s: %s", c.code, c.message)
    errors = [c for c in checks if c.level == LEVEL_ERROR]
    if errors and raise_on_error:
        raise StartupCheckError(errors)
    return checks
```

31. assistant (2026-06-21T19:14:17.825Z)

Now I need to confirm `_enforce_startup_or_exit` is actually called on the server `--server` path. Let me verify the wiring in `main()`.

32. user (2026-06-21T19:14:19.897Z)

```
origin/master:backend/main.py:845:def _enforce_startup_or_exit(config) -> None:
origin/master:backend/main.py:940:    _enforce_startup_or_exit(global_config)  # M5: fail fast
on an incoherent ACTIVE profile
```

33. user (2026-06-21T19:14:23.640Z)

```
    if args.debug_clip is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to debug a specific clip.")
            return
        run_cli_diagnose_clip(args.video_path, args.debug_clip)
        return

    # Initialize Global Config and DB
    global global_config, db_instance
    global_config = load_app_config()  # returns typed AppConfig
    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile

    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
```

34. assistant (2026-06-21T19:14:30.092Z)

All parts of the claim are now verified against origin/master. Let me do a final check on the `gs://` aspect: confirm the doc explicitly tells testers they can paste a `gs://` URI when behind the exposed/jailed config, which would conflict with the jail rejecting it. I already read DEPLOY_ALPHA.md §6 ("paste a `gs://?` video (or a path under `allowed_video_root`)") and §7 ("pre-stage to a bucket and ingest the `gs://` URI (already supported)").

Let me double-check whether there's any code path that exempts `gs://` from the jail when `allowed_video_root` is set ? i.e., a scheme check before `resolve_under_root`. There is none in `validate_input_path`/`resolve_under_root` (I read both fully). The jail unconditionally calls `os.path.realpath` on the raw string. So the discrepancy is real and confirmed by the engine's own comment at main.py:587.

Let me verify there is no test that asserts `gs://` is ALLOWED through the jail (which would mean the engine special-cases it somewhere I missed).

35. user (2026-06-21T19:14:31.526Z)

```
origin/master:tests/test_api_endpoints.py:541:    Counterweight =
test_process_hosted_mode_rejects_nonlocal_uri (gs:// -> 400 when allowed_video_root
origin/master:tests/test_api_endpoints.py:609:    """Hosted mode (allowed_video_root set) must
reject a bucket/Drive URI ? a tenant can't point
origin/master:tests/test_api_endpoints.py:614:    mp.setattr(m.global_config.security,
"allowed_video_root", str(root))
origin/master:tests/test_path_safety.py:5:from backend.utils.paths import safe_output_filename,
validate_input_path, resolve_under_root
origin/master:tests/test_path_safety.py:30:    validate_input_path(str(p))  # no allowed_root ->
only null-byte check; must not raise
origin/master:tests/test_path_safety.py:38:    validate_input_path(str(inside),
allowed_root=str(root))  # contained -> ok
origin/master:tests/test_path_safety.py:42:        validate_input_path(str(outside),
allowed_root=str(root))
origin/master:tests/test_path_safety.py:45:def
test_resolve_under_root_blocks_traversal(tmp_path):
origin/master:tests/test_path_safety.py:49:        resolve_under_root(str(root / ".." /
"escape.mp4"), str(root))
origin/master:tests/test_startup_checks.py:38:        sec["allowed_video_root"] = video_root
origin/master:tests/test_startup_checks.py:247:    and an otherwise-unsafe config (no
allowed_video_root). The strict default-OFF guarantee."""
origin/master:tests/test_startup_checks.py:336:    without the allowed_video_root jail ? BEFORE
serving a single request."""
```

36. assistant (2026-06-21T19:14:33.922Z)

The tests directly confirm the behavior. Let me read the two most decisive tests: the `gs://` rejection test and the traversal-blocking test.

37. user (2026-06-21T19:14:35.712Z)

```
    r = client.post("/api/process", json={"video_path": "local:///no/such/ls-missing.mp4"})
    assert r.status_code == 404


def test_process_hosted_mode_rejects_nonlocal_uri(env):
    """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant can't
point
    the engine at an arbitrary bucket. Pins the security contract instead of an incidental side
effect."""
    client, db, tmp_path, mp = env
    root = tmp_path / "tenant"
    root.mkdir()
    mp.setattr(m.global_config.security, "allowed_video_root", str(root))
    for bad in ("gs://b/x.mp4", "s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
        r = client.post("/api/process", json={"video_path": bad})
        assert r.status_code == 400, f"{bad} -> {r.status_code}"
    # A local file UNDER the root is still accepted (passes validation; processing mocked
elsewhere).
    under = root / "ok.mp4"
    under.write_bytes(b"hashable")
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
    r = client.post("/api/process", json={"video_path": str(under), "segmenter": "x"})
    assert r.status_code == 202


def test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
    """The CLI --video-path block (a second M2 call-site) routes a bucket URI through fetch: the
    segmenter sees a LOCAL path and the DB keeps the source URI."""
    db = Database(str(tmp_path / "cli.db"))
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path)
    monkeypatch.setattr(m, "load_app_config", lambda *a, **k: cfg)
    monkeypatch.setattr(m, "Database", lambda *a, **k: db)
```

```
    monkeypatch.setattr(m, "global_config", cfg)
```

38. user (2026-06-21T19:14:36.194Z)

```
    p = tmp_path / "vid.mp4"
    p.write_text("x")
    validate_input_path(str(p))  # no allowed_root -> only null-byte check; must not raise


def test_validate_input_path_enforces_root(tmp_path):
    root = tmp_path / "uploads"
    root.mkdir()
    inside = root / "v.mp4"
    inside.write_text("x")
    validate_input_path(str(inside), allowed_root=str(root))  # contained -> ok
    outside = tmp_path / "other.mp4"
    outside.write_text("x")
    with pytest.raises(ValueError):
        validate_input_path(str(outside), allowed_root=str(root))


def test_resolve_under_root_blocks_traversal(tmp_path):
    root = tmp_path / "r"
    root.mkdir()
    with pytest.raises(ValueError):
        resolve_under_root(str(root / ".." / "escape.mp4"), str(root))
```

39. assistant (2026-06-21T19:14:44.789Z)

All parts of the claim are now decisively verified. The test `test_process_hosted_mode_rejects_nonlocal_uri` is the smoking gun: with `allowed_video_root` set, `gs://b/x.mp4`, `s3://b/x.mp4`, and `gdrive://...` ALL return 400. This directly contradicts the DEPLOY_ALPHA.md CUJ and caveats, which tell testers to paste a `gs://` URI under the exposed/jailed config.

Let me summarize the verdict. The core claim (the three jail behaviors) is fully confirmed. But part (d) ? "does a `gs://` URI bypass the jail by design (cloud ingest) ? is that intended & safe?" ? reveals a real discrepancy in the doc that the owner must know: **the doc says testers can paste `gs://` (§6 CUJ) and `gs://` ingest is "already supported" (§7), but when `allowed_video_root` is set (which the doc itself REQUIRES for exposure in §2.2), the jail REJECTS every `gs://`/`s3://`/`gdrive://` URI with a 400.** The two instructions are mutually exclusive on the exposed alpha box.

The narrow claim about the path-jail mechanics is `confirmed`; but because the doc carries an internal contradiction about `gs://` that would make a documented CUJ fail on the exact config the doc tells the owner to deploy, I'll mark this `partial` with a blocker-severity discrepancy note for that, since the icebreaker specifically asked to "get this exactly right."

40. user (2026-06-21T19:15:09.840Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 5076 bytes): {"id": "deploy-alpha-allowed-video-root-jail", "claim": "§2.2/§7: allowed_video_root is a path-jail REQUIRED when exposed; /api/process rejects any video path outside it; upload_dir MUST sit under all
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

41. assistant (2026-06-21T19:15:14.756Z)

I need to pass the structured fields directly, not as a raw JSON string. Let me retry with proper field structure.

42. user (2026-06-21T19:15:37.865Z)

Structured output provided successfully

43. assistant (2026-06-21T19:15:48.365Z)

I've completed the verification and submitted the structured output.

Summary of findings (verified against engine `origin/master`):

The path-jail mechanics in DEPLOY_ALPHA.md §2.2/§7 are **correct and well-tested**:
- `allowed_video_root` is enforced by `backend/utils/paths.py::resolve_under_root` via `os.path.realpath` + `os.path.commonpath`, which resists `..` traversal, symlinks, absolute paths, and cross-drive escapes (test at `tests/test_path_safety.py:45`).
- `/api/process` (`backend/main.py:575-583`) validates the path FIRST and returns a clean 400 for anything outside the jail.
- The upload_dir-under-jail requirement is real: `backend/api/uploads.py:124-145` returns the assembled file's absolute path under `upload_dir`, which the SPA POSTs to `/api/process` and is re-validated by the same jail ? so an upload outside the jail 400s. The config comment at `backend/config/models.py:287-288` states this explicitly.
- The jail is genuinely REQUIRED-to-boot when exposed: `backend/config/startup.py` emits a fatal `EXPOSED_NO_VIDEO_ROOT` and `backend/main.py:845-857,940` does `sys.exit(1)`.

The one **blocker-severity discrepancy** is part (d), the gs:// question the icebreaker flagged: the doc's CUJ (§6) and §7 tell testers to paste a `gs://?` URI and call it "already supported," but the test `tests/test_api_endpoints.py:609` proves that **with `allowed_video_root` set (which §2.2 requires to expose), every `gs://`/`s3://`/`gdrive://` URI returns 400** ? because `validate_input_path` runs before `resolve_input_ref` and `os.path.realpath` treats the URI as a local path that escapes the jail. gs:// does NOT bypass the jail by design; the jail blocks it. So the two doc instructions are mutually exclusive on the alpha box. As shipped, the safe alpha flow is browser upload (kept inside the jail), not gs://.